



**UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE**

**CARRERA:**

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

**NRC: 30693**

**ASIGNATURA:**

METODOLOGÍA DE DESARROLLO DE SOFTWARE

**INTEGRANTES:**

CAYO JANINE

CHAFLA SANTIAGO

GUAYCHA ANTHONY MADRID

**EDMIR DOCENTE:**

ING. JENNY RUIZ, MGT.

## REQUISITOS FUNCIONALES

| Código | Requisito funcional   | Actor                   | Entrada                        | Salida                            |
|--------|-----------------------|-------------------------|--------------------------------|-----------------------------------|
| RF-01  | Ingreso al aplicativo | Administrador y Cliente | Usuario y contraseña           | Acceso a las opciones del sistema |
| RF-02  | Buscar cápsula        | Administrador           | Número de serie                | Información del producto          |
| RF-03  | Registrar cápsula     | Administrador           | Nombre, cantidad y precio      | Producto almacenado               |
| RF-04  | Editar cápsula        | Administrador           | Número de serie y nuevos datos | Información actualizada           |
| RF-05  | Eliminar cápsula      | Administrador           | Número de serie                | Eliminación del producto          |

## DIAGRAMA DE CASOS DE USO

@startuml

left to right direction

actor Administrador

actor Cliente

rectangle "Sistema MPG-CAPS" {

    usecase "Iniciar sesión" as UC1

    usecase "Buscar cápsula" as UC2

    usecase "Registrar cápsula" as UC3

    usecase "Editar cápsula" as UC4

    usecase "Eliminar cápsula" as UC5

    usecase "Registrar cliente" as UC6

}

### ' Relaciones del Administrador

Administrador --> UC1

Administrador --> UC3

Administrador --> UC4

Administrador --> UC5

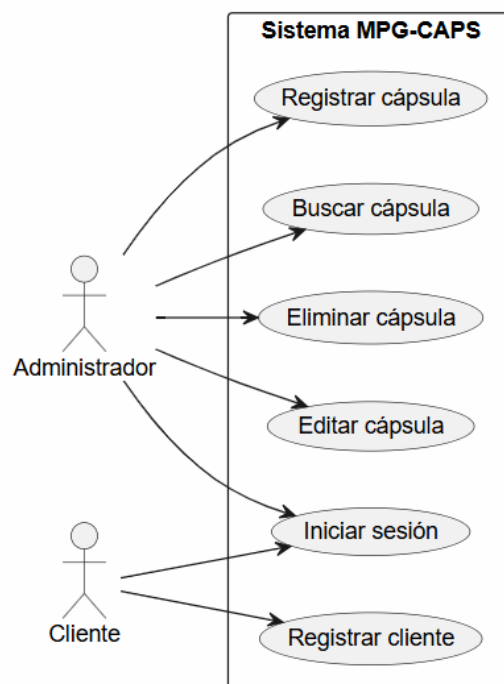
Administrador --> UC2

### ' Relaciones del Cliente

Cliente --> UC6

Cliente --> UC1

@enduml



## Interpretación

El diagrama de casos de uso representa las funcionalidades implementadas en el sistema MPG-CAPS y la interacción existente entre los actores Administrador y Cliente.

Ambos actores pueden acceder al sistema mediante el inicio de sesión utilizando sus respectivas credenciales.

El Administrador tiene acceso a la gestión de productos, cuales son los procesos de búsqueda, registro, edición y eliminación de cápsulas.

Por otra parte, el Cliente dispone de la funcionalidad de registro para almacenar sus datos personales dentro del sistema.

## 4. ESPECIFICACIÓN BREVE DE CASOS DE USO

| Caso de uso       | Actor         | Precondición                       | Flujo principal resumido                                                  | Resultado esperado   |
|-------------------|---------------|------------------------------------|---------------------------------------------------------------------------|----------------------|
| Iniciar sesión    | Administrador | Debe existir una cuenta registrada | Ingresar usuario y contraseña → validar credenciales → acceder al sistema | Acceso exitoso       |
| Buscar cápsula    | Administrador | Debe existir la cápsula            | Ingresar número de serie → consultar MongoDB → mostrar información        | Información mostrada |
| Registrar cápsula | Administrador | El número de serie no debe existir | Ingresar datos → validar → guardar en MongoDB                             | Cápsula registrada   |
| Editar cápsula    | Administrador | La cápsula debe existir            | Buscar serie → modificar datos → validar → guardar cambios                | Datos actualizados   |
| Eliminar cápsula  | Administrador | La cápsula debe existir            | Buscar serie → confirmar eliminación → eliminar registro                  | Cápsula eliminada    |

## 5. PUENTE HACIA CLASES DE ANÁLISIS (Boundary-Control-Entity)

@startuml

left to right direction

actor Administrador

boundary Vista\_Login

boundary Vista\_Admin

boundary Vista\_Producto

control Controlador\_Login

control Controlador\_Admin

control ControladorProducto

entity Administrador

entity Clientes

entity Producto

entity Repositorio\_Producto

Administrador --> Vista\_Login

Vista\_Login --> Controlador\_Login

Controlador\_Login --> Administrador

Controlador\_Login --> Clientes

Administrador --> Vista\_Admin

Vista\_Admin --> Controlador\_Admin

Controlador\_Admin --> Vista\_Producto

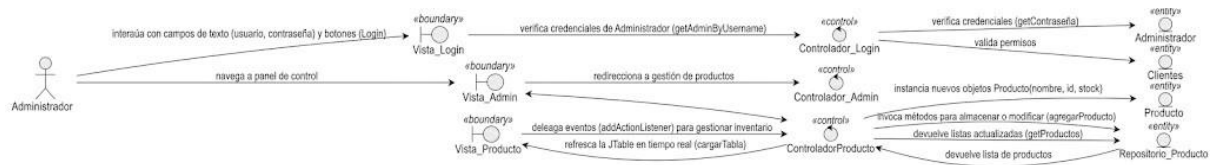
Vista\_Producto --> ControladorProducto

ControladorProducto --> Producto

ControladorProducto --> Repositorio\_Producto

@enduml

## Imagen del Diagrama



## 6. DIAGRAMA DE CLASES

@startuml

```
class Usuario{
```

```
# usuario : String
```

```
# contrasenia : String
```

```
+ ingreso(String usuario, String contrasenia)
```

```
}
```

```
class Administrador{
```

```
+ ingreso(String usuario, String contrasenia)
```

```
}
```

```
class Clientes{
```

+ ingreso(String usuario, String contrasenia)

}

class Producto{

- numeroSerie : String

- nombre : String

- cantidad : int

- precioUnitario : double

+ getNumeroSerie()

+ setNumeroSerie()

+ getNombre()

+ setNombre()

+ getCantidad()

+ setCantidad()

+ getPrecioUnitario()

+ setPrecioUnitario()

+ reducirStock(int reduccion)

}

class Repositorio\_Producto{

+ buscarCapsula(String numeroSerie)

+ editarProducto(Producto p, String numeroSerieBuscado)

+ eliminarProducto(String numeroSerie)

+ editarStock(Producto p, String numeroSerie)

+ actualizarStock(String numeroSerie, int nuevoStock)

}

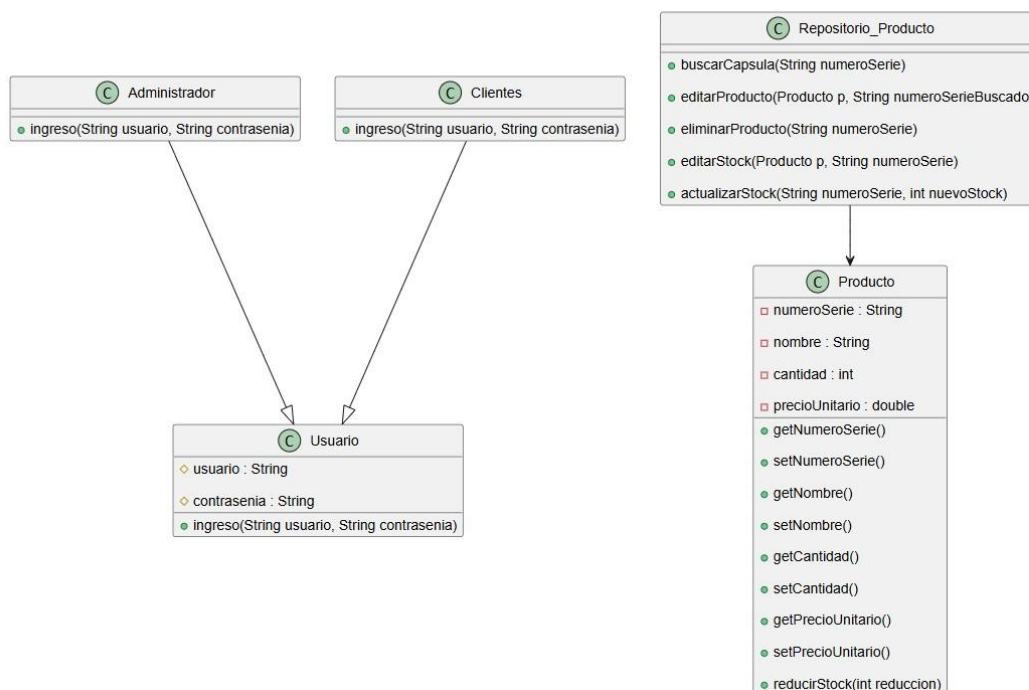
Administrador --|> Usuario

Cientes --|> Usuario

Repositorio\_Producto --> Producto

@enduml

## Imagen del Diagrama



## Interpretación

El diagrama de clases representa la estructura de las clases principales implementadas en el primer sprint del sistema MPG-CAPS.

La clase abstracta **Usuario** almacena las credenciales básicas del sistema mediante los atributos `usuario` y `contrasenia`. De esta clase heredan **Administrador** y **Clientes**, los cuales implementan el método `ingreso()` para validar el acceso al sistema.

La clase **Producto** representa las cápsulas almacenadas en el catálogo y contiene la información correspondiente a su número de serie, nombre, cantidad disponible y precio unitario, además de los métodos necesarios para acceder y modificar estos datos.

La clase Repositorio\_Producto se encarga de administrar la información almacenada en MongoDB mediante operaciones de búsqueda, edición, eliminación y actualización de los productos registrados.

#### 10. MATRIZ DE TRAZABILIDAD RF-CU

| RF    | Caso de Uso       | Actor         | Prioridad | Criterio de aceptación                   |
|-------|-------------------|---------------|-----------|------------------------------------------|
| RF-01 | Iniciar sesión    | Administrador | Alta      | Acceso correcto con credenciales válidas |
| RF-02 | Buscar cápsula    | Administrador | Alta      | Encuentra cápsulas por número de serie   |
| RF-03 | Registrar cápsula | Administrador | Alta      | Guarda cápsulas sin duplicar serie       |
| RF-04 | Editar cápsula    | Administrador | Alta      | Modifica datos correctamente             |
| RF-05 | Eliminar cápsula  | Administrador | Media     | Elimina registros existentes             |
| RF-06 | Validar datos     | Sistema       | Alta      | Impide datos vacíos o inválidos          |